

Back To Practice

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.
- If allocated, display the block number, block size, and internal fragmentation.

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int main() {
4     int m, n;
5
6     // Input for memory blocks
7     printf("Enter number of memory blocks:\n");
8     scanf("%d", &m);
9
10    // Using Variable Length Arrays (C99 standard)
11    int blockSize[m];
12    int originalBlockSize[m];
13
14    printf("Enter size of each block:\n");
15    for (int i = 0; i < m; i++) {
16        scanf("%d", &blockSize[i]);
17        originalBlockSize[i] = blockSize[i]; // Keep original sizes for the final output
18    }
19    // Input for processes
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Back To Practice

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.
- If allocated, display the block number, block size, and internal fragmentation.

```
45 // If a suitable block was found
46 if (best_idx != -1) {
47     // Deduct the process size to update the remaining space in the block
48     blockSize[best_idx] -= processSize[i];
49
50     // The fragment is now just the updated remaining size of the block
51     int fragment = blockSize[best_idx];
52
53     printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
54           i + 1, processSize[i], best_idx + 1, originalBlockSize[best_idx], fragment);
55 } else {
56     // If best_idx is still -1, no block was large enough
57     printf("Process %d of size %d is not allocated\n", i + 1, processSize[i]);
58 }
59 }
60
61 return 0;
62 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

> Test Case - 2